

Computer Organisation and Architecture

Analytical Problem-Solving Questions

C02 ASSESSMENT TOOL 1

➤ YAVANIKA S
92521258

● Question 1: 2's Complement Addition and Subtraction (+45 and -23)

Understanding the Problem

An 8-bit processor represents signed integers in 2's complement form. We must add and subtract +45 and -23, showing binary conversion, sign-bit alignment, and how the processor detects overflow from the result.

Step 1 — Binary Conversion (8-bit, 2's complement)

Decimal	Magnitude (binary)	2's Complement Representation
+45	00101101	00101101
-23	00010111	11101001 (invert 00010111 → 11101000, +1)

To form -23: write +23 in binary (00010111), invert every bit (11101000), then add 1 → 11101001.

Step 2 — Addition: (+45) + (-23)

	Binary (8-bit)	Decimal
+45	0 0 1 0 1 1 0 1	45
+ (-23)	1 1 1 0 1 0 0 1	-23
Sum	0 0 0 1 0 1 1 0	+22

Adding bit by bit generates a carry out of the sign bit (bit 7) of 1, and a carry into the sign bit that is also 1. Discarding the carry beyond the 8th bit leaves 00010110 = +22, which is the correct arithmetic result of 45 + (-23).

Step 3 — Subtraction: (+45) - (-23)

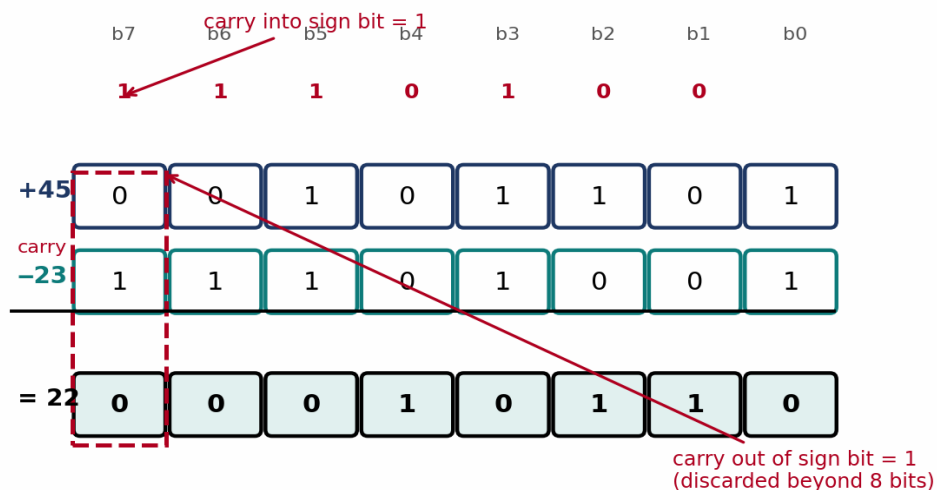
Subtraction in 2's complement hardware is performed as addition of the complement: $A - B = A + (2\text{'s complement of } B)$. Since $B = -23$, its 2's complement is simply +23 (00010111).

	Binary (8-bit)	Decimal
+45	0 0 1 0 1 1 0 1	45
+ (+23)	0 0 0 1 0 1 1 1	23
Result	0 1 0 0 0 1 0 0	+68

Here the carry into the sign bit is 0 and the carry out of the sign bit is also 0, so the two agree and the 8-bit result 01000100 = +68 is valid — matching $45 - (-23) = 68$.

Step 4 — Overflow Detection

- **Rule:** the processor compares the carry into the sign bit (C_{in}) with the carry out of the sign bit (C_{out}). If $C_{in} \neq C_{out}$, the result has overflowed the 8-bit signed range (−128 to +127) and is invalid.
- **Equivalent check:** overflow also occurs whenever two operands of the same sign produce a result of the opposite sign — an impossible outcome for that operation.
- **Here:** in both the addition and the subtraction above, $C_{in} = C_{out}$ at the sign bit, so the hardware correctly reports no overflow, and both +22 and +68 lie safely within the representable range.



Carry-in to sign bit (1) = Carry-out of sign bit (1) → NO OVERFLOW, result = +22 is valid

Fig 1.1 — Carry-in vs carry-out at the sign bit for (+45) + (−23)

Conclusion: Converting -23 by invert-and-add-1 and adding it to $+45$ gives $+22$, while adding $+23$ directly to $+45$ gives $+68$ — both within the 8-bit signed range. Because the carry into and out of the sign bit match in each case, the ALU's overflow flag stays clear and both results are trustworthy.

Question 2: Carry Look-Ahead Adder vs Ripple Carry Adder

Understanding the Problem

A ripple-carry adder is replaced with a 4-bit Carry Look-Ahead Adder (CLA) to cut delay. We must analyse how generate/propagate signals are formed, how carries are computed in advance, and why the CLA is faster.

Generate and Propagate Signals

- **Generate, $G_i = A_i \cdot B_i$:** a carry is produced at this stage regardless of any incoming carry, because both input bits are 1.
- **Propagate, $P_i = A_i \oplus B_i$:** an incoming carry would be passed straight through this stage to the next.
- **Carry equation:** $C_i = G_i + P_i \cdot C_{i-1}$. Expanding this recursively for every bit expresses every carry purely in terms of A, B and C_0 — not in terms of the previous stage's carry output — so all carries can be built by two levels of gates in parallel.

Worked Example: A = 1011 (11), B = 0011 (3)

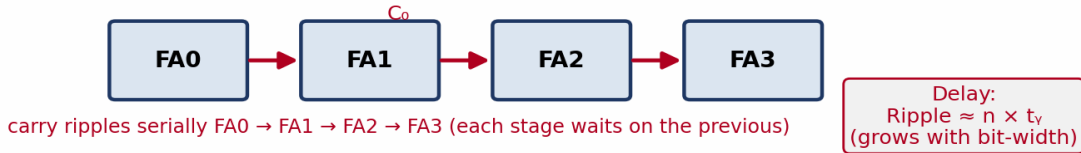
Bit i	A_i	B_i	$G_i = A_i B_i$	$P_i = A_i \oplus B_i$	$C_i = G_i + P_i C_{i-1}$	$S_i = A_i \oplus B_i \oplus C_{i-1}$
0	1	1	1	0	$C_0 = 1$	0
1	1	1	1	0	$C_1 = 1$	1
2	0	0	0	0	$C_2 = 0$	1
3	1	0	0	1	$C_3 = 0$	1

($C_3 = 0$.) Reading the carry-out C_3 with the sum bits $S_3 S_2 S_1 S_0$ gives 0 1110 = 14, which correctly equals $11 + 3$.

Time Delay: CLA vs Ripple Carry

- **Ripple carry:** stage i cannot compute its sum until C_{i-1} arrives from stage i-1, so total delay grows linearly — roughly $n \times$ (delay per full adder) for an n-bit adder.
- **Carry look-ahead:** because every C_i is expressed directly from A, B and C_0 through the G/P equations, all carries settle after only 2–3 gate delays regardless of n, and the sums follow almost immediately after.
- **Why CLA is faster in practice:** modern processors run at multi-GHz clocks where every gate delay in the critical path directly limits clock speed; removing the serial carry chain removes the single biggest bottleneck in binary addition, which is why CLA (or hybrids like carry-select/carry-skip built on the same G/P idea) is used in ALUs instead of plain ripple carry.

Ripple Carry Adder



Carry Look-Ahead Adder (CLA)

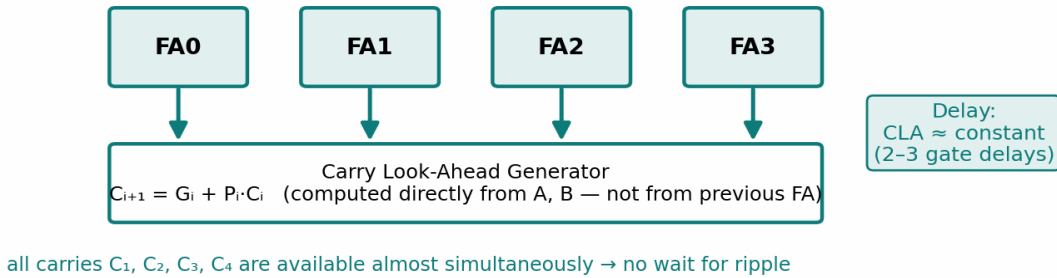


Fig 2.1 — Serial carry chain (ripple) vs precomputed carries (CLA)

Conclusion: Generate and propagate signals let every carry bit be derived directly from the inputs instead of waiting on the previous stage, collapsing an $O(n)$ ripple delay to a near-constant delay — which is exactly why high-performance processors use carry look-ahead logic over plain ripple-carry addition.

● Question 3: Booth's Algorithm: $(-6) \times (+5)$

Understanding the Problem

A processor multiplies two signed numbers, -6 and $+5$, using Booth's algorithm. We trace the accumulator (A), multiplier (Q), Booth bit (Q_{-1}) and the arithmetic shifts step-by-step.

Setup (4-bit registers)

Register	Value	Meaning
M (multiplicand)	1010	-6 in 4-bit 2's complement
Q (multiplier)	0101	$+5$ in 4-bit 2's complement
A (accumulator)	0000	initialised to 0
Q ₋₁	0	initialised to 0

- **Booth's rule (examine $Q_n Q_{n-1}$ each cycle):** '10' $\rightarrow A = A - M$ (start of a run of 1s); '01' $\rightarrow A = A + M$ (end of a run of 1s); '00' or '11' $\rightarrow$ no arithmetic operation. Every cycle ends with an arithmetic right shift of A, Q, Q_{-1} together, and the multiplicand is only added or subtracted at the boundaries of a run of 1-bits in the multiplier — not for every bit.

Step-by-Step Trace

Cycle	$Q_n Q_{n-1}$	Operation	A (after op)	After arithmetic right shift: A / Q / Q_{-1}
1	1 0	$A = A - M$	0110	0011 / 0010 / 1
2	0 1	$A = A + M$	1101	1110 / 1001 / 0
3	1 0	$A = A - M$	0100	0010 / 0100 / 1
4	0 1	$A = A + M$	1100	1110 / 0010 / —

After 4 cycles ($n = 4$ bits), the final product is the 8-bit concatenation of A and Q:

A Q (8-bit)	2's Complement Value	Check
1110 0010	-30	$(-6) \times (+5) = -30 \checkmark$

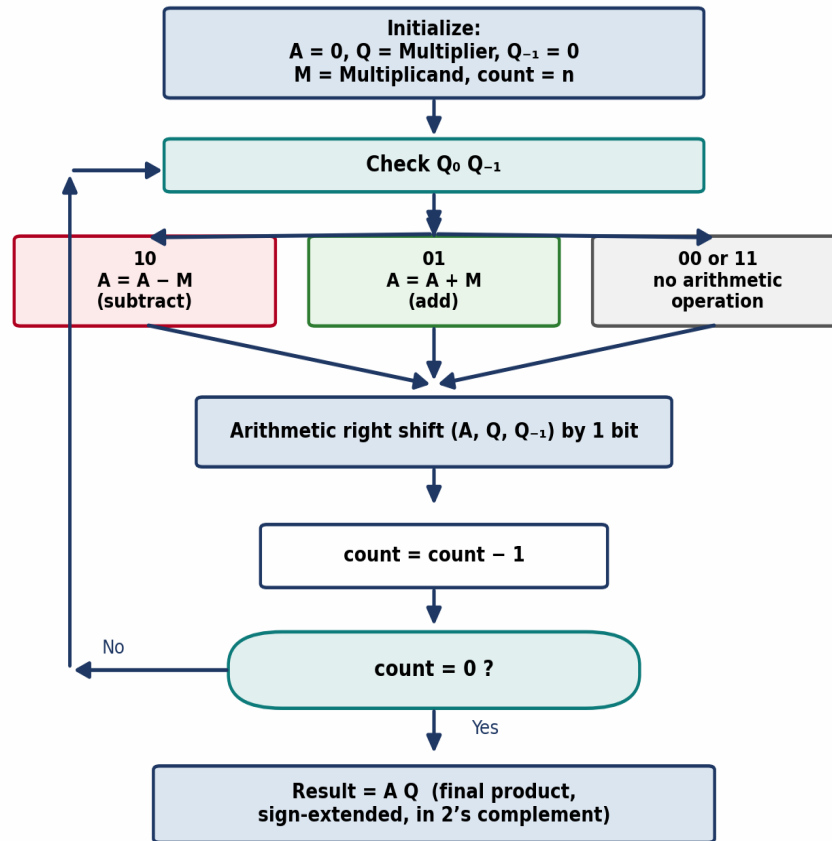


Fig 3.1 — Booth's algorithm control flow

Why Booth's Algorithm Is Efficient

- Handles signed numbers directly:** because it works entirely in 2's complement with arithmetic (sign-preserving) shifts, no separate sign-and-magnitude conversion step is needed before or after multiplying.
- Fewer add/subtract operations:** a run of consecutive 1-bits in the multiplier (e.g. 0111...110) is handled with just one subtraction at the start of the run and one addition at the end, instead of one addition per 1-bit as in traditional shift-and-add multiplication — this is exactly what the $Q_i Q_{i-1} = 00/11$ “no-operation” case exploits.

Conclusion: Booth's algorithm correctly computes $(-6) \times (+5) = -30$ using only two subtractions and two additions across four cycles, driven purely by transitions in $Q_i Q_{i-1}$. By skipping arithmetic during runs of identical bits, it directly handles signed operands and reduces the operation count compared with naive bit-by-bit multiplication.

● Question 4: Restoring and Non-Restoring Division: $13 \div 3$

Understanding the Problem

A processor divides 13 (dividend, $Q = 1101$) by 3 (divisor, $M = 0011$) using both restoring and non-restoring division. We trace intermediate remainders (A) and the quotient (Q) formed bit-by-bit over 4 iterations.

Restoring Division

- **Rule each cycle:** shift (A, Q) left by 1 bit, compute $A = A - M$. If the result is negative, restore A by adding M back and set $Q_i = 0$; otherwise keep the result and set $Q_i = 1$.

Cycle	After shift (A / Q)	A - M	Negative? Action	A / Q (final for cycle)
1	0001 / 1010	1110	Yes → restore, $Q_0=0$	0001 / 1010
2	0011 / 0100	0000	No → keep, $Q_0=1$	0000 / 0101
3	0000 / 1010	1101	Yes → restore, $Q_0=0$	0000 / 1010
4	0001 / 0100	1110	Yes → restore, $Q_0=0$	0001 / 0100

Remainder (A)	Quotient (Q)	Check
0001 = 1	0100 = 4	$4 \times 3 + 1 = 13 \checkmark$

Non-Restoring Division

- **Rule each cycle:** shift (A, Q) left by 1 bit. If A is currently non-negative, $A = A - M$; if A is currently negative, $A = A + M$ instead. Set $Q_i = 1$ if the new $A \geq 0$, else $Q_i = 0$ — with no restoring step inside the loop. After the last cycle, if A is negative, perform one final correction: $A = A + M$.

Cycle	After shift (A / Q)	Prior A sign	Operation → New A	Q	A / Q (final for cycle)
1	0001 / 1010	≥ 0	$A - M \rightarrow 1110$	0	1110 / 1010
2	1101 / 0100	< 0	$A + M \rightarrow 0000$	1	0000 / 0101
3	0000 / 1010	≥ 0	$A - M \rightarrow 1101$	0	1101 / 1010
4	1011 / 0100	< 0	$A + M \rightarrow 1110$	0	1110 / 0100

Since the final A (1110) is negative, one end-of-loop correction is applied: $A = 1110 + 0011 = 0001$.

Remainder (A)	Quotient (Q)	Check
0001 = 1	0100 = 4	$4 \times 3 + 1 = 13$ ✓ (matches restoring division)

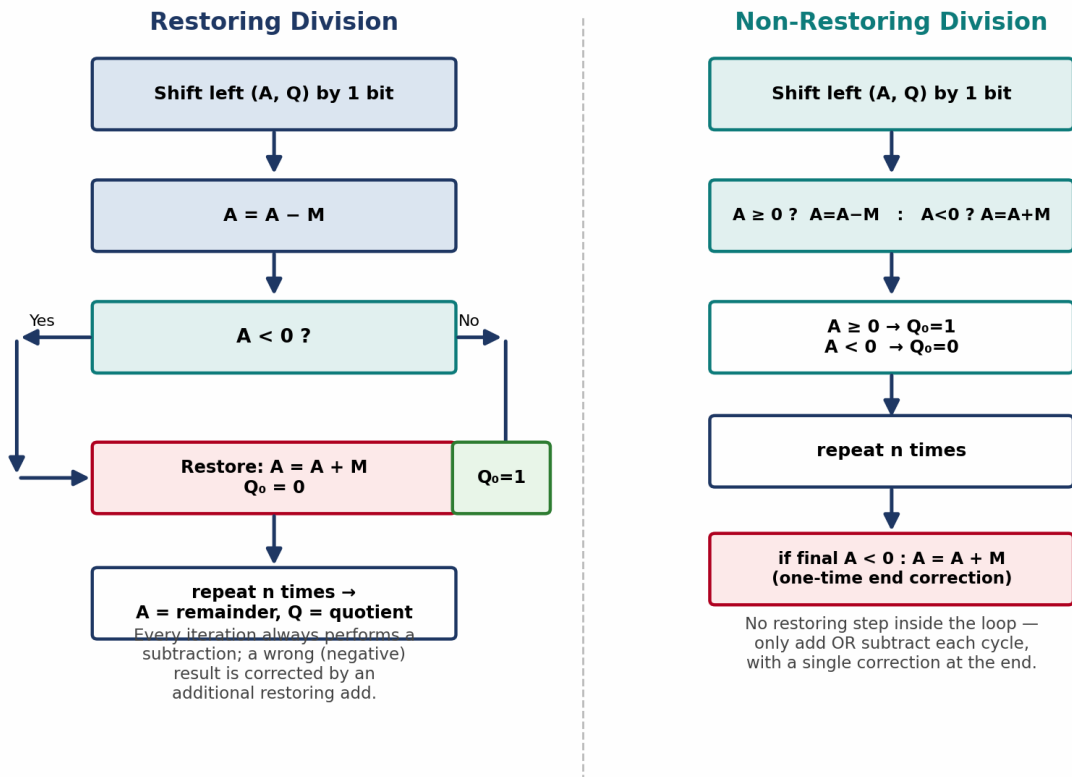
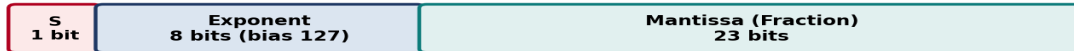


Fig 4.1 — Restoring division (subtract every cycle, restore on failure) vs non-restoring division (add or subtract, correct once at the end)

Conclusion: Both methods correctly divide 13 by 3 to give quotient 4 and remainder 1, but restoring division re-adds the divisor whenever a trial subtraction goes negative, while non-restoring division simply alternates the sign of the next operation and fixes the result once at the end — making it faster and the standard choice in modern processors.

[illegible]

Single Precision (32-bit)



Double Precision (64-bit)



$$\text{Value} = (-1)^s \times 1.\text{mantissa} \times 2^{(\text{exponent} - \text{bias})}$$

$-13.25 \rightarrow S=1$, exponent field encodes 2^3 (bias-adjusted), mantissa encodes $1.65625 \rightarrow 1.10101\dots$

Step 4 — Floating-Point Addition Example: $(-13.25) + (2.5)$

- **Represent operands:** $-13.25 = -1.10101 \times 2^3$; $2.5 = 1.01 \times 2^1$
- **Align exponents:** the smaller exponent (2^1) must be raised to match the larger one (2^3) by shifting its mantissa right by 2 bits: $1.01 \times 2^1 = 0.0101 \times 2^3$
- **Add aligned mantissas:** $-1.10101 + 0.01010 = -(1.10101 - 0.01010) = -1.01011$, so the sum is -1.01011×2^3
- **Result:** $-1.01011 \times 2^3 = -10.75$, which matches $-13.25 + 2.5 = -10.75$ exactly

Normalization, Rounding, and Precision Loss

- **Normalization:** after aligning and adding, the result may not have a leading 1 immediately left of the binary point (e.g. if the leading bits cancel out, as when subtracting two nearly equal magnitudes). The mantissa must then be shifted left until it is of the form $1.xxxx$, decrementing the exponent by one for every bit shifted.
- **Rounding:** the exact mathematical sum can need more mantissa bits than the format allows (23 for single, 52 for double). Hardware keeps extra guard, round and sticky bits during the addition and applies round-to-nearest-even to fit the result back into the fixed mantissa width.
- **Precision loss:** whenever the true result needs more bits than the mantissa can hold, the discarded low-order bits are lost permanently — this is why repeated floating-point additions (especially of a small number to a much larger one, or subtraction of nearly equal numbers) can accumulate visible error over many operations.

In this particular example the result $-10.75 = -1.01011 \times 2^3$ needed only 5 mantissa bits, well inside the 23/52-bit budget, so it is representable exactly with no rounding or precision loss.

Conclusion: -13.25 normalizes to -1.10101×2^3 , giving distinct but consistent single- and double-precision IEEE 754 encodings that differ only in exponent bias and mantissa width. Adding it to 2.5 demonstrates the three-stage process every floating-point adder performs — align

exponents, add mantissas, then re-normalize and round — and shows that precision loss only becomes a factor when the exact result needs more mantissa bits than the format provides.